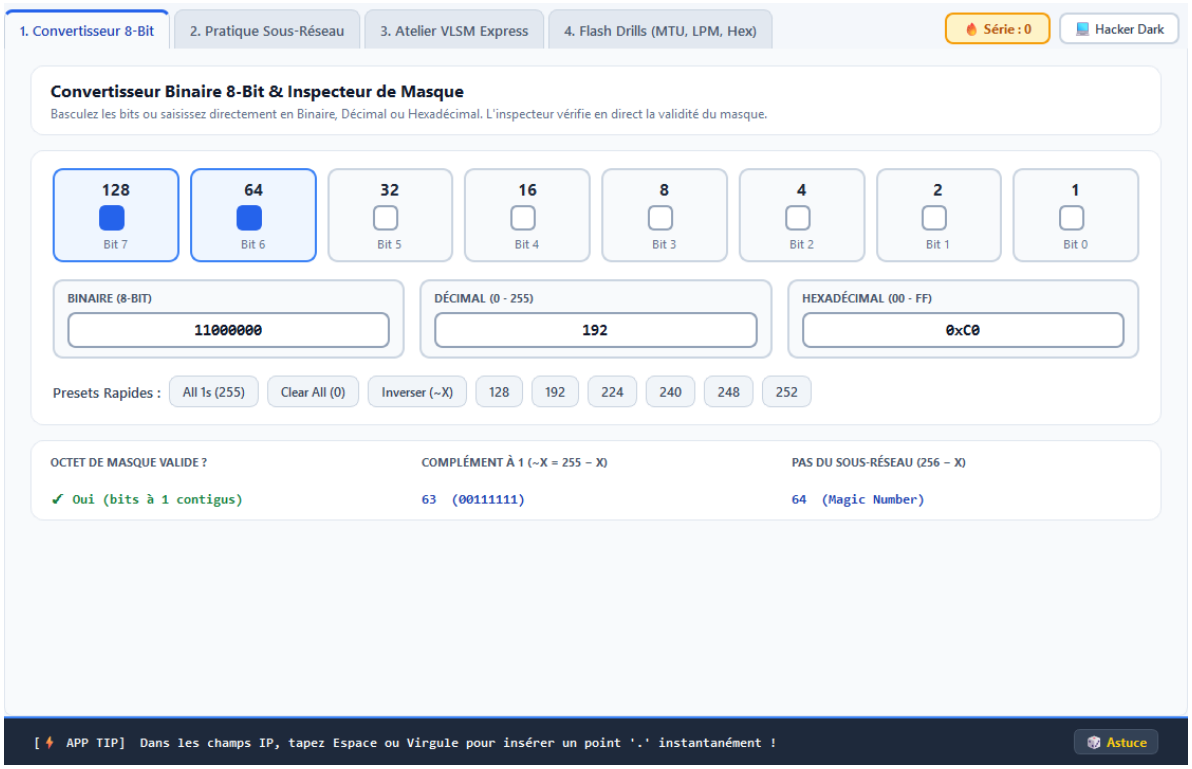
INSTITUT NATIONAL DES POSTES ET TÉLÉCOMMUNICATIONS (INPT)

Cursus Ingénieur — Filière ASEDS

IPv4 Subnet Toolkit v1.2.0

Architecture Logicielle, Ingénierie du Code Python/PyQt6 & Blueprint de Portage Java OOP

Explication complète du code source module par module : découplage Clean Architecture (`engine/` vs `ui/`), opérations binaires 32 bits, algorithmes des solveurs, événements clavier PyQt6, feuilles de style dynamiques QSS et compilation `.exe`.



Version 1.2.0 — Octobre 2026

Guide Technique & Revue de Code Complète

Contents

1 Architecture Globale : Du Monolithe v1.0.0 au Modèle Modulaire v1.2.0 **2**

1.1 Pourquoi Refactoriser l'Application en Deux Packages (`engine/` et `ui/`) ? 2

1.2 Arborescence Complète du Code Source v1.2.0 2

2 Analyse Détaillée du Moteur Réseau (`engine/`) **4**

2.1 Module 1 : `engine/ipv4_math.py` — Bitwise Math & Sous-Réseaux 32 Bits 4

2.1.1 1. Inspection d'un Octet et Piège de l'Opérateur ∼ en Python 4

2.1.2 2. Calcul Binaire 32 Bits de `Network`, `Host ID Interne` et `Broadcast` 4

2.1.3 3. Génération Aléatoire Pondérée (`generate_practice_problem`) 5

2.2 Module 2 : `engine/vlsm_solver.py` — Algorithme de Découpage VLSM 6

2.3 Module 3 : `engine/drills_solver.py` — MTU, Routage LPM & Checksum 6

3 Ingénierie de l'Interface Graphique PyQt6 (`ui/`) **8**

3.1 Saisie Clavier Intelligente : `IpLineEdit` (`ui/widgets.py`) 8

3.2 Sélecteurs Dynamiques QSS et le Cycle `unpolish` / `polish` 8

3.3 Trois Patterns PyQt6 Essentiels Utilisés dans les Onglets 9

3.3.1 1. Le Verrou Anti-Récursion `self._updating` (`ui/tab_binary.py`) 9

3.3.2 2. La Subtilité `isHidden()` vs `isVisible()` (`ui/tab_subnet.py`) 10

3.3.3 3. Timers Asynchrones Non-Bloquants (`QTimer` dans `ui/widgets.py`) 10

4 Compilation Standalone (`.exe`) & Blueprint de Portage Java OOP **11**

4.1 Fonctionnement de la Compilation PyInstaller (`converter.exe`) 11

4.2 Blueprint de Portage vers Java Orienté Objet (Cursus ASEDS INPT) 11

4.2.1 1. Traduction des `@dataclass(frozen=True)` en `record` Java 11

4.2.2 2. Le Piège Critique en Java : `int` Signé 32 Bits vs `long` 64 Bits 11

1 Architecture Globale : Du Monolithe v1.0.0 au Modèle Modulaire v1.2.0

1.1 Pourquoi Refactoriser l'Application en Deux Packages (`engine/` et `ui/`) ?

Dans la version initiale `v1.0.0`, l'intégralité du programme (1 171 lignes) résidait dans un seul fichier `converter.py` où les calculs d'adresses IP étaient exécutés directement à l'intérieur des callbacks des boutons PyQt6.

Pour la version **v1.2.0**, nous avons appliqué le patron d'architecture **Séparation des Préoccupations (*Separation of Concerns — SoC*)** :

- **Le package `engine/` (Logique Métier Pure)** : Contient 100 % des mathématiques IPv4, de l'algorithme VLSM, des solveurs de fragmentation MTU, de routage LPM et de Checksum 16 bits. **Aucun fichier de `engine/` n'importe `PyQt6`.**
- **Le package `ui/` (Couche Présentation PyQt6)** : Ne contient aucune formule mathématique complexe. Les widgets graphiques délèguent chaque calcul à `engine/`, récupèrent un objet de données immuable (`@dataclass(frozen=True)`) et mettent à jour l'affichage.

1.2 Arborescence Complète du Code Source v1.2.0

```
v1.2.0/
|-- converter.py                  # Point d'entree : instancie QApplication et
    MainWindow
|-- converter.exe                 # Executable standalone compile via PyInstaller
|-- engine/                       # MOTEUR RESEAU PUR (100% Python standard, zero
    PyQt6)
|    |-- __init__.py              # Facade d'exportation du package engine
|    |-- ipv4_math.py             # Calculs d'octet, classes A-E, RFC1918, binaire 32
    bits
|    |-- vlsm_solver.py           # Algorithme de decoupage VLSM & presets du cours
    INPT
|    |-- drills_solver.py         # Solveurs : Fragmentation MTU, Routage LPM,
    Checksum 16-bit
|    `-- tips_bank.py             # Banque de 16 astuces & gestionnaire de Serie /
    Progres
`-- ui/                           # INTERFACE GRAPHIQUE PYQT6
     |-- __init__.py
     |-- themes.py                # Feuilles de style QSS : Light Mode & Hacker Dark
         (#00FF66)
     |-- widgets.py               # IpLineEdit (auto-dot >= 26), MinecraftTipBanner,
         AdvancementToast
     |-- main_window.py           # Fenetre principale, barre d'onglets, Streak badge,
          Theme toggle
     |-- tab_binary.py            # Onglet 1 : Convertisseur 8-bit + Inspecteur de
         Masque
     |-- tab_subnet.py            # Onglet 2 : Pratique Sous-Reseau + Tiroir Binaire
         32 bits
     |-- tab_vlsm.py              # Onglet 3 : Atelier VLSM Express avec auto-
         completion
     `-- tab_drills.py            # Onglet 4 : Flash Drills (QStackedWidget : MTU, LPM
         , Checksum)
```

Avantage Majeur : Objets de Transfert Immuables (`@dataclass(frozen=True)`)

Toutes les réponses renvoyées par `engine/` (`OctetAnalysis`, `SubnetAnalysis`, `VlsmAllocation`, `VlsmProblem`, `MtuProblem`, `LpmProblem`, `ChecksumProblem`) sont déclarées avec `@dataclass(frozen=True)`. Cela garantit que l'interface graphique ne peut jamais modifier accidentellement l'état interne d'un calcul, éliminant toute une catégorie de bugs d'effets de bord.

2 Analyse Détaillée du Moteur Réseau (engine/)

2.1 Module 1 : engine/ipv4_math.py — Bitwise Math & Sous-Réseaux 32 Bits

2.1.1 1. Inspection d'un Octet et Piège de l'Opérateur ~ en Python

En Python, le type `int` a une précision arbitraire et est signé. L'opérateur d'inversion binaire `~val` calcule en réalité $-\text{val} - 1$ (complément à deux signé). Par exemple, $\sim 192 = -193$. Pour obtenir le **complément à un non signé sur 8 bits** (ou sur 32 bits pour un masque complet), il est indispensable d'appliquer un masque bit à bit `& 0xFF` (ou `& 0xFFFFFFFF`) :

```
VALID_MASK_OCTETS: Tuple[int, ...] = (255, 254, 252, 248, 240, 224, 192, 128, 0)

class IPv4MathEngine:
    @staticmethod
    def is_valid_mask_octet(val: int) -> bool:
        """Vrai si l'octet possede des bits a 1 contigus de gauche a droite."""
        return val in VALID_MASK_OCTETS

    @staticmethod
    def ones_complement_octet(val: int) -> int:
        """Retourne le complement a 1 sur 8 bits (~val & 0xFF), soit 255 - val."""
        return (~val) & 0xFF

    @staticmethod
    def block_size_octet(val: int) -> int:
        """Retourne le pas du sous-reseau / Magic Number (256 - val)."""
        return 256 - max(0, min(255, val))
```

Listing 1: Extrait de engine/ipv4_math.py : Inspection d'un octet (0..255)

2.1.2 2. Calcul Binaire 32 Bits de Network, Host ID Interne et Broadcast

Dans la méthode `analyze_subnet(ip_str, cidr)`, nous convertissons l'adresse IPv4 en un entier 32 bits via `int(ipaddress.IPv4Address(ip_str))`, puis nous construisons le masque 32 bits par décalage binaire à gauche :

```
# Construction du masque 32 bits a partir du prefixe /CIDR
mask_int = ((0xFFFFFFFF << (32 - cidr)) & 0xFFFFFFFF) if cidr > 0 else 0
wildcard_int = (~mask_int) & 0xFFFFFFFF  # Complement a 1 du masque sur 32 bits

# Formules exactes du cours INPT :
network_int   = ip_int & mask_int          # Adresse Reseau = IP ET Masque
host_id_int   = ip_int & wildcard_int      # Host ID Interne = IP ET (~Masque)
broadcast_int = network_int | wildcard_int # Broadcast = Network OU (~Masque)

# Calcul des bits empruntes (b_S) et des bits d'hotes (b_H)
if default_cidr is not None and cidr >= default_cidr:
    borrowed_bits = cidr - default_cidr
    num_subnets = 1 << borrowed_bits       # 2^b_S sous-reseaux
else:
    borrowed_bits = 0
    num_subnets = 1

host_bits = 32 - cidr
usable_hosts = max(0, (1 << host_bits) - 2) # 2^b_H - 2 hotes disponibles
```

Listing 2: Extrait de engine/ipv4_math.py : Opérations binaires 32 bits du cours

Remarquez l'utilisation de `1 << k` au lieu de `2 ** k` : en programmation système et réseau, le décalage binaire à gauche `1 << k` calcule exactement 2^k en une seule instruction processeur.

2.1.3 3. Génération Aléatoire Pondérée (`generate_practice_problem`)

Dans `v1.0.0`, les octets étaient tirés uniformément au hasard, ce qui donnait 98 % d'adresses publiques et presque jamais d'adresses `Privées`, `APIPA` ou `Loopback`. En `v1.2.0`, `generate_practice_problem()` utilise une distribution de probabilité contrôlée (`random.random()`) :

- 12 % de chance de générer une adresse **Loopback** (`127.x.x.x`, Classe A).
- 12 % de chance de générer une adresse **APIPA** (`169.254.x.x`, Classe B).
- 34 % de chance de générer une adresse **Privée RFC 1918** (`10.x.x.x`, `172.16..31.x.x`, ou `192.168.x.x`).
- 42 % de chance de générer une adresse **Publique** (Classe A, B ou C).

De plus, le `cidr` est toujours tiré dans l'intervalle $[\text{default_cidr}, 30]$ pour que la formule du cours $b_S = \text{CIDR} - \text{default_cidr}$ soit toujours applicable.

2.2 Module 2 : `engine/vlsm_solver.py` — Algorithme de Découpage VLSM

Le solveur VLSM implémente fidèlement la méthode vue en TD :

```
class VlsmSolver:
    @staticmethod
    def smallest_host_bits(hosts_needed: int) -> int:
        """Trouve la plus petite puissance h >= 2 telle que 2^h - 2 >= hosts_needed."
            ""
        if hosts_needed < 1:
            return 2
        h = max(2, math.ceil(math.log2(hosts_needed + 2)))
        while (1 << h) - 2 < hosts_needed:
            h += 1
        return h

    @classmethod
    def solve(cls, base_network_str: str, base_cidr: int, needs: List[int], title:
        str) -> VlsmProblem:
        base_net = ipaddress.IPv4Network(f"{base_network_str}/{base_cidr}", strict=
            False)
        current_ip_int = int(base_net.network_address)

        # Etape 1 : Tri strictement decroissant des besoins
        sorted_needs = sorted(needs, reverse=True)
        allocations: List[VlsmAllocation] = []

        # Etape 2 : Allocation sequentielle sans trou
        for idx, need in enumerate(sorted_needs, start=1):
            h = cls.smallest_host_bits(need)
            cidr = 32 - h
            block_size = 1 << h
            subnet = ipaddress.IPv4Network((current_ip_int, cidr), strict=False)
            ...
            # Avancer le pointeur d'adresse de la taille exacte du bloc (2^h)
            current_ip_int += block_size
```

Listing 3: Extrait de engine/vlsm_solver.py : Dimensionnement et allocation séquentielle

Pourquoi avancer simplement `current_ip_int += block_size` fonctionne-t-il toujours ? Parce que `sorted_needs` est trié par ordre décroissant, la suite des tailles de blocs $2^{h_1}, 2^{h_2}, 2^{h_3}$ est décroissante ($h_1 \geq h_2 \geq h_3$). Or, en arithmétique binaire, la somme Base $+ 2^{h_1}$ (où Base est alignée sur /24, donc multiple de 256) est nécessairement divisible par 2^{h_2} dès lors que $h_2 \leq h_1 \leq 8$. Ainsi, chaque sous-réseau tombe exactement sur une frontière de masque valide !

2.3 Module 3 : `engine/drills_solver.py` — MTU, Routage LPM & Checksum

```
# 1. Alignement de la charge utile MTU sur un multiple de 8 octets :
header_len = ihl * 4
orig_data = max(0, total_length - header_len)
max_data = ((mtu - header_len) // 8) * 8

# 2. Somme de controle IPv4 (16 bits) avec rabattement iteratif de la retenue :
@staticmethod
def solve_checksum(words_hex: List[str], title: str = "") -> ChecksumProblem:
    cleaned_words = [w.strip().upper().replace("0X", "").zfill(4) for w in words_hex]
    raw_sum = sum(int(w, 16) for w in cleaned_words)

    carried = raw_sum
    while carried > 0xFFFF:
        carried = (carried & 0xFFFF) + (carried >> 16)

    checksum = (~carried) & 0xFFFF
```

```
    return ChecksumProblem(title, cleaned_words, raw_sum, carried, checksum)
```

Listing 4: Extrait de engine/drills_solver.py : Fragmentation MTU et Checksum 16 bits

Détail subtil d'ingénierie : Dans `solve_checksum`, nous utilisons une boucle `while carried > 0xFFFF:` et non un simple `if`. En effet, si `raw_sum` valait par exemple `0x1FFFF`, un seul rabattement donnerait `0xFFFF` + `0x0001` = `0x10000`, qui dépasse à nouveau `0xFFFF` ! La boucle `while` garantit que le résultat tient toujours sur 16 bits.

3 Ingénierie de l'Interface Graphique PyQt6 (ui/)

3.1 Saisie Clavier Intelligente : IpLineEdit (ui/widgets.py)

Pour offrir une saisie ultra-fluide sans avoir à chercher la touche . sur le clavier, la classe `IpLineEdit` hérite de `QLineEdit` et redéfinit la méthode virtuelle `keyPressEvent(self, event: QKeyEvent)` :

```
class IpLineEdit(QLineEdit):
    def keyPressEvent(self, event):
        key = event.key()
        text = self.text()
        pos = self.cursorPosition()

        # 1. Espace ou Virgule insere un '.' ou saute par-dessus un '.' existant
        if key in (Qt.Key.Key_Space, Qt.Key.Key_Comma):
            if pos > 0 and text[pos - 1] not in ". ":
                if text.count(".") < 3:
                    self.insert(".")
                    return
            elif pos < len(text) and text[pos] == ".":
                self.setCursorPosition(pos + 1)
                return
            return

        # 2. Laisser QLineEdit traiter la frappe normale du caractere
        super().keyPressEvent(event)

        # 3. Lookahead mathematique : Auto-Dot apres 3 chiffres OU 2 chiffres >= 26
        if event.text().isdigit():
            cur_text = self.text()
            cur_pos = self.cursorPosition()
            if cur_pos == len(cur_text):
                last_dot = cur_text.rfind(".")
                last_space = cur_text.rfind(" ")
                sep = max(last_dot, last_space)
                octet = cur_text[sep + 1:] if sep != -1 else cur_text
                dots_count = cur_text.count(".") + cur_text.count(" ")

                if dots_count < 3 and octet.isdigit():
                    val = int(octet)
                    if len(octet) == 3 or (len(octet) == 2 and val >= 26):
                        self.insert(".")
```

Listing 5: Code complet de IpLineEdit dans ui/widgets.py

Pourquoi vérifier `event.text().isdigit()` après `super().keyPressEvent(event)` ? Si l'utilisateur appuie sur la touche `Backspace` pour effacer un point ., `event.text().isdigit()` vaut `False`. Ainsi, l'auto-dot ne se redéclenche jamais pendant un effacement, permettant de corriger librement sa saisie !

3.2 Sélecteurs Dynamiques QSS et le Cycle unpolish / polish

Au lieu de modifier le style de chaque widget en Python avec des couleurs codées en dur (ce qui empêcherait de basculer proprement entre **Light Mode** et **Hacker Dark Mode**), nous utilisons des **propriétés dynamiques Qt** associées à des sélecteurs d'attributs dans `ui/themes.py` :

```
#bitBox[active="true"] {
    background-color: #00220A;
    border: 2px solid #00FF66;
}
#choiceBtn[selected="true"] {
    background-color: #00FF66;
    border: 1.5px solid #00FF66;
    color: #000000;
```



Figure 1: Rendu du thème **Hacker Dark** (`ui/themes.py`) sur l'onglet VLSM grâce aux propriétés dynamiques QSS.

```
}
#autoSummaryBadge[solved="true"] {
    border: 1.5px solid #00FF66;
    color: #00FF66;
    background-color: #00220A;
}
```

Listing 6: Extrait de ui/themes.py : Sélecteurs d'attributs QSS

Dans le code Python (`tab_binary.py`, `tab_subnet.py`, `tab_vlsm.py`, `tab_drills.py`), lorsqu'un état change, on applique le pattern suivant :

```
badge.setProperty("solved", True)
badge.style().unpolish(badge)
badge.style().polish(badge)
```

En Qt, `setProperty()` modifie l'attribut interne du `QObject`, mais le moteur de rendu CSS ne réévalue pas l'apparence tant qu'on n'appelle pas `unpolish()` suivi de `polish()`.

3.3 Trois Patterns PyQt6 Essentiels Utilisés dans les Onglets

3.3.1 1. Le Verrou Anti-Récursion `self._updating` (`ui/tab_binary.py`)

Dans l'onglet 1, modifier le champ Décimal met à jour les 8 cases à cocher, le champ Binaire et le champ Hexadécimal. Pour empêcher que le changement d'une case à cocher ne rappelle à son tour `on_checkbox_toggled()` en boucle infinie, `apply_value()` encadre toutes les mises à jour avec `self._updating = True` dans un bloc `try...finally`, et ne réécrit pas dans le champ `source` en cours de frappe (préservant la position du curseur de l'utilisateur).

3.3.2 2. La Subtilité `isHidden()` vs `isVisible()` (`ui/tab_subnet.py`)

Pour ouvrir/fermer le tiroir binaire 32 bits, nous utilisons :

```
def toggle_binary_drawer(self):
    vis = self.drawer_card.isHidden()
    self.drawer_card.setVisible(vis)
```

En PyQt6, `widget.isVisible()` retourne `False` si l'onglet parent n'est pas l'onglet actuellement affiché à l'écran, tandis que `widget.isHidden()` reflète l'état propre du tiroir indépendamment de l'onglet actif.

3.3.3 3. Timers Asynchrones Non-Bloquants (`QTimer` dans `ui/widgets.py`)

- `MinecraftTipBanner` utilise un `QTimer` périodique (`setInterval(12000)`) pour faire tourner les 16 astuces toutes les 12 secondes sur la boucle d'événements Qt sans jamais créer de thread secondaire ni utiliser `time.sleep()`.
- `AdvancementToast` utilise un `QTimer` *single-shot* (`setSingleShot(True)`, `3200 ms`) qui masque automatiquement la bannière `Progrès débloqué !` au bout de 3,2 secondes.

4 Compilation Standalone (.exe) & Blueprint de Portage Java OOP

4.1 Fonctionnement de la Compilation PyInstaller (converter.exe)

L'exécutable autonome `v1.2.0/converter.exe` est généré via la commande :

```
python -m PyInstaller --onefile --noconsole --name "converter" converter.py
```

- `-onefile` : Empaquette l'interpréteur Python 3, les bibliothèques dynamiques Qt6 (`Qt6Core.dll`, `Qt6Gui.dll`, `Qt6Widgets.dll`) et nos packages `engine/` et `ui/` dans une archive compressée unique (`CArchive`).
- `-noconsole` : Utilise le bootloader Windows graphique (`runw.exe`) pour lancer directement la fenêtre PyQt6 sans ouvrir d'invite de commandes noire en arrière-plan.

4.2 Blueprint de Portage vers Java Orienté Objet (Cursus ASEDS INPT)

Grâce à l'isolation totale du dossier `engine/`, vous pouvez porter ce projet en **Java 17/21** (avec JavaFX ou Swing) en traduisant directement chaque module :

4.2.1 1. Traduction des `@dataclass(frozen=True)` en `record` Java

```java
package ma.inpt.aseds.subnet.engine;

public record OctetAnalysis(
    int value,
    String binaryStr,
    String hexStr,
    boolean isValidMaskOctet,
    int onesComplement,
    String onesComplementBinary,
    int blockSize
) {}
```

Listing 7: Équivalent Java direct de OctetAnalysis et VlsmAllocation

4.2.2 2. Le Piège Critique en Java : `int` Signé 32 Bits vs `long` 64 Bits

En Python, `0xFFFFFFFF` vaut +4 294 967 295. En Java, le type `int` est signé sur 32 bits : `0xFFFFFFFF` vaut `-1`, et l'opérateur » propage le bit de signe 1 à gauche ! Lors du portage Java de `IPv4MathEngine` et `DrillsSolver`, appliquez ces deux règles :

```java
// Regle 1 : Utiliser long (64 bits) avec le suffixe L pour les masques 32 bits
long maskInt = (cidr > 0) ? ((0xFFFFFFFFL << (32 - cidr)) & 0xFFFFFFFFL) : 0L;
long wildcardInt = (~maskInt) & 0xFFFFFFFFL;
long networkInt = ipInt & maskInt;
long hostIdInt = ipInt & wildcardInt;
long broadcastInt = networkInt | wildcardInt;

// Regle 2 : Utiliser le decalage non signe >>> pour le Checksum 16 bits
int carried = rawSum;
while (carried > 0xFFFF) {
    carried = (carried & 0xFFFF) + (carried >>> 16);
}
int checksum = (~carried) & 0xFFFF;
```

Listing 8: Opérations binaires IPv4 sûres en Java